# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

### *Experiments*

**COURSE NAME:** Computer Networks

**COURSE CODE:** CSA07

**COURSE OUTCOME COVERED**

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

**Assessment Tool Weightage:** 40%

**Code of Conduct:**

I **Kongara Jogesh (192525035)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

*Kongara Jogesh*

# Experiments

**QUESTIONS: 4 Total Marks: 40**

**Name:** Kongara Jogesh **Reg No:** 192525035

## Question 1: TCP Three-Way Handshake (10 Marks)

*Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection.*

### *Aim*

To capture live network traffic using Wireshark and analyze the TCP three-way handshake mechanism that is used to establish a reliable connection between a client and a server before any application data is exchanged.

### *Tools Used*

- Wireshark (Packet capture and protocol analyzer)
- Web browser / terminal (to initiate a TCP connection, e.g., opening a website or using telnet/curl)
- A system connected to the internet or LAN

### *Procedure*

- Open Wireshark and select the active network interface (Wi-Fi/Ethernet).
- Start the packet capture.
- Open a browser and navigate to a website (e.g., http://example.com), or run a command such as "curl http://example.com" from the terminal, so that a fresh TCP connection is initiated.
- Apply the display filter "tcp.flags.syn==1 || tcp.flags.ack==1" or simply "tcp" in Wireshark to isolate the handshake packets.
- Stop the capture and locate the first three packets exchanged between the client and the server.

### *Observation - The Three Handshake Packets*

| Step | Direction | Flag Set | Purpose / Role |
|---|---|---|---|
| 1 | Client → Server | **SYN** | The client sends a segment with the SYN flag set and an Initial Sequence Number (ISN). This packet requests the opening of a connection and synchronizes the client's sequence number with the |

| | | | |
|---|---|---|---|
| | | | server. |
| 2 | Server → Client | **SYN, ACK** | The server replies with both the SYN and ACK flags set. The ACK acknowledges the client's ISN (Ack = client ISN + 1), and the SYN carries the server's own ISN, requesting synchronization in the reverse direction. |
| 3 | Client → Server | **ACK** | The client sends a final segment with only the ACK flag set, acknowledging the server's ISN (Ack = server ISN + 1). At this point the connection is officially ESTABLISHED and data transfer can begin. |

***How the Handshake Establishes a Reliable Connection***

- Sequence number synchronization: Each side generates its own Initial Sequence Number (ISN) and communicates it to the other side, so both hosts agree on a starting point for byte-level tracking of the data stream.
- Mutual confirmation: The ACK flag in packets 2 and 3 proves that each side has actually received the other's SYN, so neither side starts sending data until both are ready to receive.
- Connection state agreement: Before the handshake, both sides are in the CLOSED/LISTEN state. After SYN, SYN-ACK, and ACK are exchanged, both move to the ESTABLISHED state, guaranteeing that data exchanged afterward has a known, agreed-upon path and sequence numbering.
- Reliability foundation: Because both sequence numbers are known in advance, TCP can later detect lost, duplicated, or out-of-order segments and retransmit or reorder them correctly.

***Result***

The TCP three-way handshake (SYN → SYN-ACK → ACK) was successfully captured and analyzed in Wireshark. It was observed that this exchange allows both the client and the server to agree on initial sequence numbers and confirm each other's readiness, thereby establishing a reliable, connection-oriented communication channel before any application data is transmitted.

## Question 2: DNS Query over UDP – Header Comparison (10 Marks)

*Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability.*

***Aim***

To capture a DNS query in Wireshark, observe that it uses UDP as the transport protocol, and compare the structure and size of the UDP header with the TCP header.

***Procedure***

- Open Wireshark and start capturing on the active interface.
- Clear the local DNS cache (optional) and open a browser or run "nslookup www.google.com" to trigger a fresh DNS lookup.
- Apply the display filter "dns" in Wireshark to isolate DNS packets.
- Select a DNS query packet and expand the "User Datagram Protocol" section in the packet details pane to view the UDP header fields.
- For comparison, expand a TCP packet captured earlier (from Question 1) and view its "Transmission Control Protocol" header fields.

***Header Size Comparison***

| Parameter | UDP Header | TCP Header |
|---|---|---|
| Minimum header size | 8 bytes (fixed) | 20 bytes (can extend up to 60 bytes with options) |
| Fields present | Source Port, Destination Port, Length, Checksum | Source Port, Destination Port, Sequence Number, Acknowledgment Number, Header Length, Flags, Window Size, Checksum, Urgent Pointer, Options |
| Sequence number | Absent | Present (tracks byte order) |
| Acknowledgment number | Absent | Present (confirms receipt) |
| Connection setup | None (connectionless) | Requires 3-way handshake |

***Observation***

On expanding the DNS query packet in Wireshark, the transport layer section showed a User Datagram Protocol header containing only four fields — Source Port, Destination Port, Length, and Checksum — occupying just 8 bytes. In contrast, the TCP header captured for the handshake in Question 1 contained additional fields such as Sequence Number, Acknowledgment Number, Flags, and Window Size, making it 20 bytes or larger. Notably, the UDP header had no Sequence Number or Acknowledgment Number fields at all.

***Impact of UDP's Lightweight Design on Speed and Reliability***

- Speed: Because UDP does not require a connection setup (no handshake) and carries a much smaller header, a DNS query can be sent and answered in a single round trip, which is ideal for short, latency-sensitive lookups.

- Lower overhead: The absence of sequence and acknowledgment numbers means the sender does not need to maintain per-connection state, buffers, or timers, reducing processing overhead on both the client and the DNS server.
- Reduced reliability: Since UDP does not acknowledge receipt or retransmit lost segments, a dropped DNS query or response is not automatically recovered by the transport layer; the application (the resolver) must implement its own timeout and retry logic.
- No ordering guarantee: UDP does not guarantee that packets arrive in order, but for a single, small DNS query/response pair this is rarely an issue since the entire exchange typically fits in one packet each way.

***Result***

It was observed that DNS queries use UDP with a compact 8-byte header, unlike TCP's larger and more complex header that includes sequence and acknowledgment numbers. This lightweight design significantly reduces latency and overhead, making UDP well-suited for fast operations like DNS resolution, at the cost of built-in reliability.

## Question 3: Packet Loss – TCP Retransmission vs UDP (10 Marks)

*Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP.*

***Aim***

To simulate packet loss during active TCP and UDP sessions and observe, using Wireshark, how each protocol responds to the loss.

***Procedure***

- Start a TCP-based transfer (e.g., download a large file over HTTP, or use FTP/SCP) while Wireshark is capturing on the interface.
- Simulate packet loss by briefly disabling the network adapter, disconnecting the Wi-Fi for a few seconds, or using a tool such as "tc" (Linux) / "clumsy" (Windows) to artificially drop packets, then reconnect.
- In parallel, start a UDP-based transfer (e.g., a video/voice call, or a simple UDP file-transfer utility) and repeat the same brief disconnection.
- Stop the capture and apply the filter "tcp.analysis.retransmission" to isolate retransmitted TCP segments.
- Observe the UDP stream during the same time window using the filter "udp" and check the sequence of packets in "Statistics > Conversations".

### *Observation*

#### *TCP Behaviour*

- Packets sent during the disconnection window were not acknowledged by the receiver.
- Wireshark flagged several frames as "[TCP Retransmission]" or "[TCP Fast Retransmission]", visible in red/black coloring in the packet list.
- The sender's Retransmission Timeout (RTO) expired for unacknowledged segments, causing TCP to resend the exact same segment(s) with the same sequence number after the link was restored.
- TCP's congestion window was reduced after the loss event, and throughput temporarily dropped (visible in the Wireshark I/O graph) before ramping back up.

#### *UDP Behaviour*

- Packets sent during the disconnection window were simply lost — Wireshark showed a gap in the sequence of UDP datagrams with no retransmitted duplicates.
- There was no mechanism at the transport layer to detect or recover the missing datagrams; if the application (e.g., a media player) noticed the gap, it was up to the application layer to conceal it (e.g., a brief audio glitch) or ignore it.
- The UDP stream simply continued with new data once connectivity resumed, without ever recovering the lost datagrams.

#### *Why This Makes TCP Reliable but Slower*

- Reliability: TCP maintains sequence numbers and acknowledgments for every byte sent, and a retransmission timer for unacknowledged data. If an acknowledgment is not received within the RTO, TCP assumes the segment was lost and retransmits it, guaranteeing eventual delivery of all data in the correct order.
- Cost of reliability: Detecting loss requires waiting for a timeout (or duplicate ACKs) before retransmitting, and successful delivery must be confirmed before the sender's window can advance. This waiting period, combined with congestion-window reduction after a loss event, adds latency and reduces throughput.
- UDP's speed trade-off: UDP has no such waiting, no acknowledgments, and no congestion control, so it keeps sending at the application's rate regardless of loss. This keeps latency low and is why UDP is preferred for real-time applications, but it means any lost packet is simply gone unless the application layer handles recovery.

#### *Result*

The experiment confirmed that TCP detects and retransmits lost segments to guarantee reliable, in-order delivery, whereas UDP has no built-in mechanism to detect or recover lost datagrams.

This is why TCP is considered reliable but comparatively slower, while UDP is faster but not guaranteed to deliver every packet.

## Question 4: DNS Resolution – Query Types and Timing (10 Marks)

*Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.*

### *Aim*

To capture and analyze DNS traffic in Wireshark while resolving a domain name, identify the type of DNS query and its corresponding response, and measure the resolution time.

### *Procedure*

- Open Wireshark and start capturing on the active network interface.
- Flush the local DNS resolver cache and run a lookup command, e.g., "nslookup www.example.com" or "dig www.example.com A", or simply type the domain into a browser address bar.
- Apply the display filter "dns" in Wireshark.
- Locate the DNS query packet and inspect the "Queries" section to identify the query type (A, AAAA, MX, etc.) and the domain name requested.
- Locate the matching DNS response packet (same transaction ID) and inspect the "Answers" section for the resolved IP address or mail server record.
- Note the timestamp of the query packet and the corresponding response packet; Wireshark's "Time" column (or the "[Time since request]" field inside the response packet details) gives the resolution delay directly.

### *Observation*

| Query Type | Meaning | Example Response | Typical Resolution Time |
|---|---|---|---|
| **A** | Requests the IPv4 address of the domain | A single IPv4 address, e.g., 93.184.216.34 | A few milliseconds (cached resolver) to ~100-200 ms (uncached, remote server) |
| **AAAA** | Requests the IPv6 address of the domain | A single IPv6 address, e.g., 2606:2800:220:1:248:1893:25c8:1946 | Comparable to A record queries, often sent in parallel with the A query |
| **MX** | Requests the mail exchange server(s) | Mail server hostname with a preference value, e.g., 10 | Similar order of magnitude as A |

| | | | |
|---|---|---|---|
| | for the domain | mail.example.com | queries |

In the captured trace, the query packet showed "Standard query 0xXXXX A www.example.com" in the Queries section, and the matching response packet (identified by the same transaction ID) contained "Standard query response ... A 93.184.216.34" in the Answers section. Using the timestamps of the two packets, the round-trip resolution time was measured as the difference between the response packet's arrival time and the query packet's send time — typically a few milliseconds when using a local/cached resolver, and up to a couple of hundred milliseconds when the query had to travel to an external authoritative server.

***Why DNS Typically Uses UDP***

- Small message size: Most DNS queries and responses are small enough to fit in a single UDP datagram (traditionally under 512 bytes, and now up to the configured EDNS0 size), so there is no need for TCP's segmentation and stream management.
- Speed: Since UDP does not require a connection handshake, a DNS lookup can typically be completed in a single round trip, which is critical because a DNS lookup usually precedes every other network operation (e.g., loading a webpage) and any added delay is directly felt by the user.
- Low overhead for high query volume: DNS servers handle a very large number of simultaneous queries; UDP's connectionless, stateless nature avoids the memory and processing overhead of maintaining thousands of individual TCP connections.
- Built-in application-layer reliability: The DNS protocol itself includes a transaction ID and a retry/timeout mechanism at the resolver (client) level, so it can tolerate occasional UDP packet loss by simply re-sending the query, without needing TCP's reliability features.
- Fallback to TCP when needed: DNS does use TCP when the response is too large for a single UDP datagram (e.g., large DNSSEC responses) or for zone transfers between DNS servers, showing that UDP is the default for its speed advantage while TCP is kept as a reliable fallback.

***Result***

DNS traffic was successfully captured and analyzed in Wireshark. The query type (A/AAAA/MX) and its corresponding response were identified using the DNS transaction ID, and the resolution time was measured from the packet timestamps. It was concluded that DNS predominantly uses UDP because it offers faster, lower-overhead resolution for the small, latency-sensitive request-response exchanges that DNS lookups typically involve.

# RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| **Understanding of Concepts** | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| **Experimental Design** | 30 | Well-planned, systematic implementation with optimal solution | Correct implementation with minor issues | Basic implementation with errors or inefficiencies | Incorrect or incomplete implementation |
| **Use of Tools** | 20 | Effective and advanced use of tools/technologies with proper configuration | Appropriate use of tools with correct execution | Limited or inconsistent use of tools | Incorrect or minimal use of tools |
| **Analysis of Results** | 15 | Insightful analysis with accurate interpretation and validation of results | Good analysis with reasonable interpretation | Basic analysis with limited insights | Poor or incorrect analysis of results |
| **Documentation** | 10 | Clear, well-structured documentation with diagrams, code, and results | Organized documentation with necessary details | Basic documentation with missing details | Poor or incomplete documentation |